MODULE 12 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Scheduling

How Kubernetes picks a node, and every tool you have to steer it.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 12 of 35

admin@sthithapragna.com

The scheduler answers one question per Pod: **which node?** It decides from **requests and rules**, not actual usage.

How the scheduler places a Pod

01
Watch
A new Pod appears with no node.

02
Filter
Drop nodes that cannot run it: resources, taints, selectors, affinity.

03
Score
Rank the rest: preferences, spreading, balance.

04
Bind
Write the winning node's name into the Pod.

05
Run
That node's kubelet starts it.

ON THE EXAM

No node survives the filter: the Pod stays Pending, with a FailedScheduling event giving the reasons.

Requests versus limits

REQUESTS

Decide placement

- Read by the scheduler
- A reservation on the node
- Sum of requests must fit allocatable
- Also the Pod's guaranteed share

LIMITS

Decide punishment

- Enforced by the kernel via cgroups
- A ceiling while running
- Over CPU limit: throttled
- Over memory limit: OOMKilled

THE TIEBREAKER

Requests decide placement. Limits decide punishment. Set only limits, and requests are copied from them.

Declaring resources

```
containers:
- name: api
  image: quayside/booking-api:2.4.1
  resources:
    requests: {cpu: 250m, memory: 256Mi}
    limits:   {cpu: "1", memory: 512Mi}
```

5 250m is a quarter of a CPU. Mi is mebibytes, 1024-based.

6 "1" is one whole CPU core or vCPU.

THE UNITS

CPU in cores or millicores; memory in Mi and Gi. 1000m equals one CPU.

The scheduler reads requests, not usage

- **A quiet node can be full.** If its capacity is already requested, nothing more fits.
- **A busy node can look empty.** Pods with no requests reserve nothing.
- **The message to recognise:** 0/6 nodes are available: 6 Insufficient memory.
- **Right-size requests** from real usage, so reservations match reality.

> ON THE EXAM
>
> 'Insufficient cpu' while dashboards show idle nodes: requests are reserved, not used.

Quality of service classes

CLASS	WHEN	UNDER NODE MEMORY PRESSURE
Guaranteed	Every container sets CPU and memory, with requests equal to limits	Least likely to be evicted
Burstable	Some requests or limits set, but not Guaranteed	In between
BestEffort	No requests or limits at all	Most likely to be evicted

ON THE EXAM

Kubernetes assigns the class automatically from your resource settings. You never set it directly.

Node-pressure eviction, precisely

- **The kubelet evicts Pods** when a node runs short of memory, disk or process IDs.
- **It ranks by:** usage above requests first, then Pod priority, then how far above.
- **QoS predicts the result:** BestEffort always exceeds its zero requests; Guaranteed rarely does.
- Different from a drain, which uses the Eviction API, and from an OOM kill of one container.

> ON THE EXAM
>
> Exam shorthand: BestEffort first, Burstable next, Guaranteed last.

Selectors and affinity attract. Taints repel. Spread constraints balance.

admin@sthithapragna.com

The placement toolkit

TOOL	SET ON	EFFECT
nodeSelector	Pod	Only nodes with exactly these labels
Node affinity	Pod	Required or preferred node label rules, with operators
Pod affinity / anti-affinity	Pod	Near, or away from, other Pods, per topology
Taints	Node	Repel Pods that do not tolerate them
Tolerations	Pod	Permission to land on a tainted node
Topology spread	Pod	Even out Pods across zones or nodes

ON THE EXAM

Everything is set on the Pod except taints, which are set on the node.

Node affinity: required and preferred

```
1  affinity:
2    nodeAffinity:
3      requiredDuringSchedulingIgnoredDuringExecution:
4        nodeSelectorTerms:
5        - matchExpressions:
6          - {key: topology.kubernetes.io/zone, operator: In, values: [zone-a, zone-b]}
7      preferredDuringSchedulingIgnoredDuringExecution:
8      - weight: 50
9        preference:
10         matchExpressions:
11         - {key: node.quayside.io/disk, operator: In, values: [nvme]}
```

3 A hard rule. No matching node means Pending.

7-8 A soft rule: adds to a node's score, weighted 1 to 100.

IgnoredDuringExecution: if node labels change later, running Pods are not moved.

Required versus preferred

REQUIRED...

A filter

- Nodes that fail are removed
- No match: Pod stays Pending
- Use for true constraints
- Licensing, compliance, hardware

PREFERRED...

A score

- Nodes that match score higher
- No match: placed elsewhere
- Use for nice-to-haves
- Faster disks, same zone as cache

THE TIEBREAKER

If the rule must never be broken, required. If breaking it is merely worse, preferred.

Pod affinity and anti-affinity

- **Anti-affinity** keeps replicas apart: never two booking API Pods on one node.
- **Affinity** keeps Pods together: put the cache in the same zone as the API.
- **topologyKey** sets the unit: kubernetes.io/hostname for nodes, topology.kubernetes.io/zone for zones.
- **Required anti-affinity with more replicas than nodes** leaves the extras Pending.

> ON THE EXAM
>
> Affinity rules about Pods need a topologyKey: near or apart, measured by node or by zone.

Taint effects

EFFECT	NEW PODS WITHOUT A TOLERATION	PODS ALREADY RUNNING
`NoSchedule`	Not scheduled here	Stay
`PreferNoSchedule`	Avoided if possible	Stay
`NoExecute`	Not scheduled here	Evicted, after any tolerationSeconds

ON THE EXAM

Control plane nodes carry a NoSchedule taint, which is why ordinary Pods never land there.

Tainting a node, and tolerating it

```
$ kubectl taint nodes gpu-1 dedicated=ml:NoSchedule

# in the ML Pod's spec:
tolerations:
- {key: dedicated, operator: Equal, value: ml, effect: NoSchedule}

$ kubectl taint nodes gpu-1 dedicated=ml:NoSchedule-
```

1. key=value:effect, on the node.
5. The matching toleration, on the Pod.
7. A trailing minus removes the taint.

Taints versus node affinity

TAINT + TOLERATION

Keeps others out

- Set on the node
- Repels Pods without a toleration
- A toleration permits, never attracts
- Tolerating Pods may still go elsewhere

NODE AFFINITY

Pulls yours in

- Set on the Pod
- Attracts it to matching nodes
- Does not stop other Pods arriving
- Other Pods may fill the node

THE TIEBREAKER

Dedicated nodes need both: a taint to keep others out, affinity to bring yours in.

Topology spread constraints

- **Even out replicas** across zones or nodes, rather than just keeping them apart.
- **maxSkew:** the largest allowed difference in Pod count between any two zones.
- **whenUnsatisfiable:** DoNotSchedule is a hard rule; ScheduleAnyway is a preference.
- **Quayside:** booking API, maxSkew 1 across zones, so losing a zone loses a third.

> ON THE EXAM
>
> Anti-affinity says never together. Spread says evenly. Spread scales past one Pod per node.

Priority and preemption

- **A PriorityClass** gives Pods a number. Higher numbers are scheduled first.
- **Preemption:** if a high-priority Pod cannot fit, lower-priority Pods may be evicted to make room.
- **Built in:** system-cluster-critical and system-node-critical, for core components.
- Priority also feeds node-pressure eviction ranking, after usage above requests.

> ON THE EXAM
>
> 'Critical Pods must always find room, even if others make way' is a PriorityClass.

Special cases

- **nodeName** skips the scheduler entirely and pins a Pod to one node. Rarely wise.
- **schedulerName** hands a Pod to a custom or second scheduler.
- **GPUs and devices:** Dynamic Resource Allocation, stable since 1.34, requests devices through ResourceClaims.
- The older approach, extended resources such as nvidia.com/gpu, still works.

> ON THE EXAM
>
> With nodeName set, filters, taints and resource checks by the scheduler are all bypassed.

Currency check: in-place Pod resize

- **Stable since Kubernetes 1.35:** change a running Pod's CPU and memory without recreating it.
- **Through the resize subresource,** for example kubectl patch pod … --subresource resize.
- **resizePolicy per resource:** NotRequired, the default, applies it live; RestartContainer restarts.
- **Old answer, now wrong:** 'resources are immutable, you must delete and recreate the Pod'.

> ON THE EXAM
>
> If the node lacks room for the new requests, the resize waits rather than moving the Pod.

Eight placement decisions. In each, ask: **attract, repel, balance**, or is it about **requests**?

> New booking API Pods stay Pending with the event: 0/6 nodes are available: 6 Insufficient memory.
>
> Monitoring shows every node using only about 40% of its memory.

What is the most likely explanation?

A The monitoring system is under-reporting memory use

B The kubelet on every node has stopped reporting

C Existing Pods' memory requests already reserve the nodes

D The new Pods' memory limits are set too low

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The monitoring system is under-reporting memory use

Nothing suggests monitoring is wrong; the gap is expected.

B The kubelet on every node has stopped reporting

Then nodes would be NotReady, not 'Insufficient memory'.

C Existing Pods' memory requests already reserve the nodes

The scheduler counts requests, not usage. Reservations fill nodes.

D The new Pods' memory limits are set too low

Low limits cause OOM kills later, not Pending now.

THE SIGNAL

'Insufficient' on idle nodes: requests are reserved, not used.

> When a node runs short of memory, Quayside wants the payment worker to be among the last Pods evicted.
>
> It must be done through the payment worker's own resource settings.

Which settings achieve this?

A Requests equal to limits for CPU and memory, everywhere

B No requests or limits at all, so it reserves no capacity

C Low requests with much higher limits, to allow bursting

D Deploy it into the kube-system namespace instead

◆ PAUSE AND DECIDE

Which settings achieve this?

A Requests equal to limits for CPU and memory, everywhere

Guaranteed QoS: usage rarely exceeds requests, so it is ranked last.

B No requests or limits at all, so it reserves no capacity

BestEffort: the most likely to be evicted.

C Low requests with much higher limits, to allow bursting

Burstable: usage above requests puts it early in the queue.

D Deploy it into the kube-system namespace instead

Namespaces play no part in eviction ranking.

THE SIGNAL

'Last to be evicted', from resource settings alone: Guaranteed.

> The booking API runs 3 replicas on a 5-node cluster. A node failure must never take out more than one replica.
>
> So no two booking API Pods may ever share a node.

Which rule enforces this?

A. A nodeSelector that lists three chosen nodes

B. A NoSchedule taint on every node in the cluster

C. Setting replicas equal to the number of nodes

D. Required pod anti-affinity with topologyKey hostname

◆ PAUSE AND DECIDE

Which rule enforces this?

A. A nodeSelector that lists three chosen nodes
Pins to three nodes; still allows two Pods on one of them.

B. A NoSchedule taint on every node in the cluster
Repels everything, including the booking API itself.

C. Setting replicas equal to the number of nodes
Changes the count; says nothing about placement.

D. Required pod anti-affinity with topologyKey hostname
A hard rule: no two matching Pods per node.

THE SIGNAL

'Never two on one node' is required anti-affinity by hostname.

> Quayside adds two GPU nodes. Only machine-learning Pods may run on them, and every machine-learning Pod must run on them.
>
> Ordinary Pods must never land on the GPU nodes.

What configuration achieves both requirements?

A. A toleration on ML Pods, with no taint on the GPU nodes

B. Taint the GPU nodes; give ML Pods toleration and affinity

C. A nodeSelector on the ML Pods only, and no taint anywhere

D. A high PriorityClass for every machine-learning Pod

◆ PAUSE AND DECIDE

What configuration achieves both requirements?

A. A toleration on ML Pods, with no taint on the GPU nodes
With no taint, a toleration does nothing at all.

B. Taint the GPU nodes; give ML Pods toleration and affinity
Taint keeps others out; affinity pulls ML Pods in.

C. A nodeSelector on the ML Pods only, and no taint anywhere
ML Pods go there, but ordinary Pods can still land there too.

D. A high PriorityClass for every machine-learning Pod
Priority affects order and preemption, not which nodes.

THE SIGNAL

Two requirements, others out and ours in: taint plus affinity.

> GPU nodes carry the taint dedicated=ml:NoSchedule. The team adds a matching toleration to the ML Pods, and nothing else.
>
> Some ML Pods land on the GPU nodes, but many land on ordinary nodes.

Why?

A A toleration permits the node; it does not attract to it

B The taint effect should have been NoExecute, not NoSchedule

C The scheduler ignores tolerations on Deployment Pods

D The taint should have been placed on the Pods instead

◆ PAUSE AND DECIDE

Why?

A A toleration permits the node; it does not attract to it

Tolerating Pods may go anywhere. Add node affinity to pull them in.

B The taint effect should have been NoExecute, not NoSchedule

NoExecute evicts running Pods; it attracts nothing either.

C The scheduler ignores tolerations on Deployment Pods

Tolerations work for every Pod, however it was created.

D The taint should have been placed on the Pods instead

Taints belong on nodes; tolerations belong on Pods.

THE SIGNAL

A toleration is permission, not direction.

> An engineer adds the taint maintenance=true:NoExecute to node-4. Several running Pods on node-4 have no toleration for it.
>
> None of them sets tolerationSeconds, and no other change is made.

What happens to those running Pods?

A Nothing, until each Pod is next restarted

B Only newly created Pods are kept away

C They are evicted from node-4 straight away

D node-4 is removed from the cluster

◆ PAUSE AND DECIDE

What happens to those running Pods?

A Nothing, until each Pod is next restarted

That describes no taint effect at all.

B Only newly created Pods are kept away

That is NoSchedule. NoExecute also acts on running Pods.

C They are evicted from node-4 straight away

NoExecute evicts running Pods that do not tolerate it.

D node-4 is removed from the cluster

Taints never delete nodes.

THE SIGNAL

NoExecute is the effect that evicts running Pods.

> The web front end has required pod anti-affinity by hostname. The cluster has 3 nodes, and the team scales web to 4 replicas.
>
> Three Pods run; one stays Pending. Spreading is desirable, but not essential.

What is the best fix?

A Increase maxSurge on the Deployment

B Change the rule to preferred anti-affinity

C Remove the Pods' resource requests

D Give the web Pods a higher PriorityClass

◆ PAUSE AND DECIDE

What is the best fix?

A Increase maxSurge on the Deployment
maxSurge only matters during rollouts.

B Change the rule to preferred anti-affinity
Spread when possible, share a node when needed.

C Remove the Pods' resource requests
Requests are not the blocker; the rule is.

D Give the web Pods a higher PriorityClass
Preemption cannot satisfy anti-affinity either.

THE SIGNAL

Hard rule, not enough nodes, and the rule is only 'desirable': make it preferred.

> During a ticket sale, each booking API Pod needs more CPU. The number of Pods must stay the same, and Pods must not restart.
>
> The cluster runs Kubernetes 1.37, and the nodes have spare capacity.

What should the team do?

A Delete each Pod so it is recreated with more CPU

B Edit the Deployment template's CPU and let it roll out

C Add a HorizontalPodAutoscaler to add more Pods

D Resize the running Pods through the resize subresource

◆ PAUSE AND DECIDE

What should the team do?

A Delete each Pod so it is recreated with more CPU

Recreates every Pod: exactly what they must avoid.

B Edit the Deployment template's CPU and let it roll out

A template change is a rollout: new Pods replace old ones.

C Add a HorizontalPodAutoscaler to add more Pods

Changes the Pod count, which must stay the same.

D Resize the running Pods through the resize subresource

In-place resize, stable since 1.35: new CPU, same Pods.

THE SIGNAL

'More CPU, same Pods, no restart' is in-place resize.

Module 12 in five lines

- Filter, score, bind. **Requests decide placement; limits decide punishment.**
- QoS: **Guaranteed** (requests = limits), **Burstable**, **BestEffort** (none). BestEffort goes first.
- **Affinity attracts, taints repel, spread balances.** Required filters; preferred scores.
- **A toleration permits, never attracts.** NoExecute evicts running Pods.
- **PriorityClass** for preemption; **in-place resize** is stable since 1.35.

That completes Kubernetes Fundamentals. Next domain: **Container Orchestration**, starting with the network.